An Artifact-Backed Empirical Study of Local RAG Execution on Constrained Unified Memory Devices

Chaoran Wang[1]

[1]College of Science and Information, Qingdao Agricultural University, Qingdao, China, Email: chaoranw47@gmail.com

Abstract

Local Retrieval-Augmented Generation (RAG) on Apple Silicon Unified Memory Architecture (UMA) is constrained not only by retrieval quality, but by the shared execution budget consumed by model weights, KV cache, retrieved context, and operating-system activity. This paper studies a local RAG execution path that combines GGUF deployment, TurboQuant retrieval compression, and KV-aware context pruning on a Mac mini M4 with 16GB unified memory. Within the `Falcon-7B` family, migrating from `transformers + MPS` to a quantized `GGUF` route reduces TTFT from `77.6 s` to `123.7 ms`; across deployable configurations, `Llama-3-8B-Q + GGUF` serves as the preferred practical reference at `51.2 ms`. In the keep-ratio scan, `keep_ratio = 0.6` reduces TTFT from `5.83 s` to `0.87 s` while decode throughput remains in the `18-19 TPS` range. Under concurrent load, the optimized path lowers P95 TTFT and wall time, while also exposing a stability cost at higher concurrency. Stage-wise latency decomposition shows that prompt-side and retrieval-side optimizations recover deployability on the tested device, but the dominant residual cost migrates to generation, especially decode. These findings are reported as an artifact-linked systems study on a constrained UMA deployment path and support the view that local RAG optimization is best treated as shared-budget systems co-design rather than isolated module acceleration.

Keywords: Local RAG; GGUF; llama.cpp; TurboQuant; Context Compression; Unified Memory Architecture

1 Introduction

Retrieval-Augmented Generation (RAG) has become a standard way to ground Large Language Models (LLMs) with private or domain-specific knowledge [2]. As RAG systems move from cloud infrastructure to consumer Apple Silicon devices with constrained Unified Memory Architecture (UMA), however, the relevant bottleneck changes. On these platforms, end-to-end latency is shaped not only by retrieval quality, but by shared memory pressure across model weights, KV cache activations, retrieved context, and operating-system processes [5]. Under such constraints, a pipeline that is accurate in principle may still fail as a practical interactive system.

The resulting research gap is therefore not simply "how to retrieve better," but how to architect a local RAG execution path that remains jointly deployable, measurable, and quality-preserving under strict hardware boundaries. Our central systems observation is that once retrieval-side cost is partially reduced, the dominant latency budget migrates toward generation. On a 16GB UMA device, unmanaged execution can instead collapse into paging-dominated behavior before a stable local baseline is even established.

This study frames that problem as *collaborative optimization*: joint tuning of retrieval compression, prompt-budget control, and backend selection under a shared hardware budget. The term does not imply runtime bidirectional communication between modules; it refers to deployment-time coordination across sequential stages whose combined footprint determines whether the full path remains usable. We analyze four tightly coupled components: (1) GGUF-based quantized deployment for hardware-aware inference, (2) TurboQuant as a retrieval-side compression lever [10], (3) KV-aware context pruning to mitigate prefill-side memory pressure, and (4) telemetry-driven orchestration for isolating stage-wise bottlenecks. The thesis of the paper is that local RAG performance on constrained UMA hardware is best understood as a systems co-design problem in which retrieval savings, prompt-budget control, and backend choice compete within the same execution budget.

Methodologically, this work emphasizes three principles tailored for systems research. First, claims are strictly anchored to empirical benchmark results documented in project artifacts. Second, negative results—such as the performance collapse of unoptimized backends—are treated as first-order evidence for identifying hardware boundaries. Third, we explicitly distinguish engineering significance (e.g., recovering interactive responsiveness) from statistical significance to avoid overstating observations drawn from heterogeneous system benchmarks.

The contribution of this work is threefold:

1. We formulate local RAG execution on constrained UMA hardware as a collaborative optimization problem span-

ning backend selection, prompt-budget control, and retrieval compression.

2. We provide an artifact-linked empirical study using real-time kernel probes to track virtual memory paging activity and relate backend choice to hardware saturation.

3. We show through stage-wise latency decomposition that, once deployability is recovered, the dominant optimization target migrates from retrieval/prefill toward generation, especially decode.

Accordingly, the problem can be stated as follows: under unified-memory and bandwidth constraints, how can a local RAG pipeline achieve the best practical trade-off between answer quality and latency? This work does not propose a novel retriever or a new serving engine in isolation; instead, it reports a deployment-oriented systems study of how these components interact under a shared UMA budget. The remainder of this paper develops this framework through quantitative execution analysis, paging evidence, and a deployment-oriented interpretation of bottleneck migration.

2 Related Work

2.1 Local inference backends and quantized deployment

Traditional local LLM inference often relies on the `transformers` library with MPS or CUDA acceleration. This route is attractive because it aligns with the mainstream Python ecosystem and provides a direct path from public model checkpoints to runnable inference. Under constrained memory conditions, however, these routes may suffer from substantial framework overhead and inefficient memory traffic. Recent serving systems have made clear that memory management is itself a first-order systems problem in LLM deployment rather than an implementation detail [8].

In contrast, lightweight C++ inference engines such as `llama.cpp` target deployability under tighter hardware budgets [4]. Their practical value comes less from introducing a new model family than from changing the execution substrate: quantized weights, memory-mapped model loading, and backend-aware kernels make it possible to restore local execution in environments where heavier routes struggle. The broader Apple Silicon ecosystem has likewise emphasized that unified-memory execution requires attention to runtime efficiency rather than only model quality [5]. The maturation of GGUF and K-Quant formats therefore matters not only as a packaging convenience, but as an enabling condition for studying 7B- to 8B-class local models on 16GB UMA devices [3].

2.2 Retrieval compression and approximate search

As the size of local corpora grows, retrieval-side memory traffic becomes increasingly relevant. In many RAG discussions, retrieval is treated primarily as a quality problem: the central question is which retriever or reranker retrieves the most useful passages. In constrained local systems, however, retrieval also has an execution cost. Index size, vector precision, and distance-computation strategy all influence how much memory traffic the system must generate before prompt construction even begins.

Techniques such as asymmetric distance computation and low-bit quantization reduce retrieval cost by replacing floating-point distance computation with cheaper approximate operations [10]. This is especially relevant when retrieval and generation share the same hardware budget. In that setting, compressed retrieval is not merely an indexing trick; it becomes a systems lever that changes how much memory and latency budget remain available for downstream prefill and decode. The present paper adopts this systems interpretation rather than treating retrieval compression as an isolated ANN benchmark problem.

2.3 Sparse retrieval and fusion in practical RAG

Production-oriented RAG systems rarely depend on a single retrieval signal. Survey work on RAG has repeatedly shown that practical systems often combine lexical and semantic evidence rather than relying on one scorer alone [2]. BM25 remains a widely used sparse baseline because of its robustness on lexical matches, identifiers, and domain-specific terminology [9]. Reciprocal Rank Fusion (RRF) is often adopted as a lightweight late-fusion strategy for combining heterogeneous retrieval outputs without introducing a heavy learned reranker [1].

This work does not claim novelty in sparse retrieval or fusion itself. Instead, these methods form part of the practical retrieval substrate whose gains eventually encounter a generation-side bottleneck. This distinction is important because it separates the present contribution from a standard retrieval-improvement narrative: the focus here is not on inventing a new fusion rule, but on analyzing what happens once the retrieval stack has already become "good enough" relative to the execution bottleneck.

2.4 KV cache management and context optimization

KV cache consumption is one of the dominant costs during local inference. Prior work on efficient LLM serving has shown that memory layout and token management can strongly shape generation efficiency [8]. Once prompts become retrieval-augmented rather than purely conversational, this issue becomes even more pronounced because the back-

end must process not only user input but also a potentially large body of inserted evidence.

Context optimization strategies can therefore be interpreted as an indirect form of KV management. By reducing prompt length before inference starts, a system limits the prefill burden and constrains the amount of activation state that must be carried into decode. Budget-aware prompt compression methods such as LLMLingua make this cost explicit by treating prompt length as an inference-time resource allocation problem rather than only as a prompting problem [7]. In RAG settings, such compression can be made more task-aware by incorporating retrieval-side salience into generation-side pruning decisions. The contribution of the current paper lies in placing this idea into an execution study: rather than proposing a new theoretical KV policy, it measures how keep-ratio-based context pruning changes TTFT, throughput, and end-to-end latency under an actual constrained deployment path.

2.5 Position of this work

The present paper sits at the intersection of these lines of work. It draws on practical RAG retrieval foundations [2, 9, 1], adopts retrieval compression as a memory-saving systems lever [10], and treats backend efficiency and memory management as first-class deployment concerns [8, 5]. Its novelty is therefore not a new retriever, a new quantizer, or a new serving engine in isolation. Rather, it contributes an artifact-linked systems study showing how backend choice, retrieval compression, and context pruning interact once local RAG is constrained by unified-memory execution.

3 Methods

3.1 Collaborative execution path

The proposed system follows a retrieval-compression-inference execution path. In this manuscript, *collaborative optimization* means joint tuning under a shared hardware budget rather than runtime inter-module communication. Instead of treating retrieval, context construction, and inference as isolated stages, the system evaluates them as a single performance-critical pipeline whose combined footprint determines whether local execution remains usable. In practical terms, the retrieval module controls how much information is passed forward, potentially through sparse scoring or lightweight fusion [9, 1]; the context compressor determines how much of the retrieved content is preserved for prefill; and the inference backend determines whether the remaining workload is executable within the latency budget.

This design reflects a systems view of local RAG. On cloud infrastructure, retrieval and generation are often discussed separately because sufficient memory and accelerator capacity hide many of the interactions between stages. On a 16GB unified-memory device, however, these stages are tightly coupled. A larger retrieved context increases prompt assembly cost, raises prefill latency, and enlarges the KV footprint that the backend must sustain during decode. Likewise, an aggressive retrieval-side compression policy may release memory pressure yet also reduce recall, which then affects answer quality. The execution path is therefore organized as a chain of dependent decisions rather than as a set of independently optimized modules.

Operationally, each query passes through four steps. First, the retriever accepts a user query and returns a candidate set under either the uncompressed or TurboQuant-compressed path. Second, the context manager consumes that candidate set and outputs the final prompt body after keep-ratio-based pruning. Third, the inference backend accepts the final prompt and produces the token stream under a selected deployment route. Fourth, telemetry records retrieval, scoring/compression, TTFT, decode throughput, and wall-clock behavior so that improvements can be attributed to retrieval, prefill, or decode rather than to aggregate wall time alone. This decomposition is important because the paper's central claim is not merely that one route is faster, but that local RAG optimization must be diagnosed and improved stage by stage.

3.2 GGUF and llama.cpp deployment path

The system adopts `llama-cpp-python` as the main inference wrapper and uses GGUF quantization as the model packaging format [4, 3]. This choice is motivated by the need to reduce deployment overhead on Apple Silicon. Compared with the standard `transformers + MPS` route, the GGUF path provides a lighter-weight execution backend and a more hardware-friendly model loading strategy.

The practical motivation for this backend migration is twofold. First, the tested hardware is constrained not only by total memory capacity but also by unified-memory bandwidth. A route that introduces large framework overhead or inefficient memory traffic can therefore become unusable even if the raw model size nominally fits on device. Second, the benchmark goal is to study end-to-end responsiveness rather than only theoretical model support. GGUF and `llama.cpp` are attractive in this setting because they reduce deployment friction through quantized weights, memory-mapped loading, and a thinner execution stack, thereby making it possible to study whether the remaining bottleneck is model execution itself or merely the surrounding runtime path.

Methodologically, the backend comparison follows a simple protocol. The `transformers + MPS` route is first retained as a negative-result baseline to establish infeasibility on the tested device. The GGUF route is then treated as the reference execution substrate once that infeasibility is visible. All subsequent pruning and retrieval experiments are interpreted within that restored substrate rather than as standalone backend benchmarks. This is important because the paper does not argue that GGUF is universally superior across all environments; it argues that, on the tested

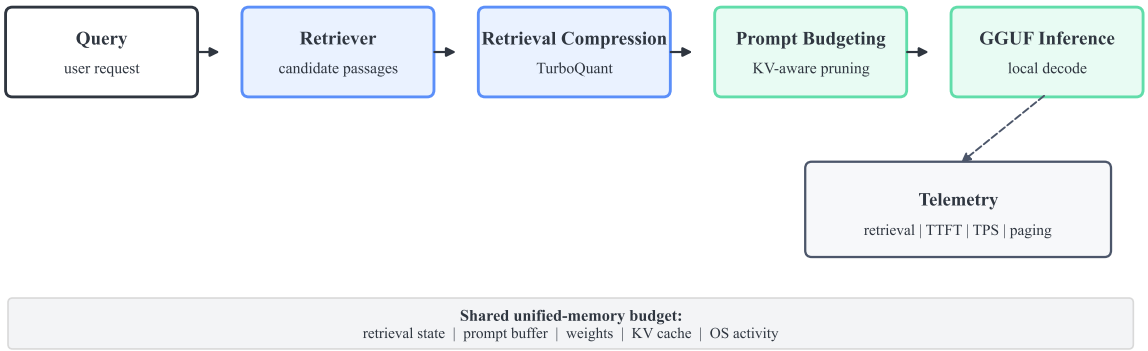

Figure 1: Stage-wise execution path of the proposed local RAG pipeline on the tested unified-memory device. Retrieval state, prompt construction, model weights, KV cache, and background system activity compete for the same memory budget. The figure summarizes the measurement scope of this study by linking retrieval compression, context pruning, local GGUF inference, and telemetry within a single constrained execution path.

UMA platform, GGUF is the first route that restores a usable baseline from which later optimizations can be meaningfully evaluated.

3.3 TurboQuant retrieval compression

TurboQuant is utilized as a high-efficiency retrieval-side compression layer to minimize the memory-bus traffic during similarity search [10]. The practical objective is to shrink the retrieval footprint before the generation phase begins, thereby releasing bandwidth for the KV-cache and model weight transfers. In this paper, TurboQuant is evaluated as a retrieval-footprint control lever rather than as a standalone ANN contribution.

To mitigate the impact of high-variance outlier dimensions—common in transformer-based embeddings—the system applies a randomized orthogonal rotation using the Fast Walsh-Hadamard Transform (FWHT) before scalar quantization. By leveraging the bit-reversal butterfly circuit, the rotation complexity is strictly bounded at $\mathcal{O}(d \log d)$, achieving sub-linear scalability relative to embedding dimensionality d. Scoring is then executed via Asymmetric Distance Computation (ADC) using 4-bit quantized centroids with a 1-bit residual correction to ensure retrieval fidelity. This compression is not merely a storage optimization; it is a systems lever that shifts retrieval from a bandwidth-bound operation toward a compute-bound operation, increasing the resource ceiling for downstream generation.

This distinction matters for interpretation. A retrieval-compression method may appear unattractive if judged only by recall loss relative to an uncompressed index. In a constrained local deployment setting, however, the retrieval layer also competes for memory and latency budget with prompt construction and model execution. The relevant question is therefore not simply whether compression reduces retrieval quality, but whether the reduced memory and latency footprint creates enough downstream benefit to justify its use in a larger execution pipeline. Accordingly, the experiments report both retrieval-centric metrics (`Recall@10`, MAE, Max Error) and pipeline-centric consequences such as end-to-end latency shifts.

3.4 KV-aware context pruning

The context compressor employs an adaptive pruning strategy to minimize the prefill burden. Our core hypothesis is that on UMA devices, reducing the input sequence length provides an order-of-magnitude TTFT improvement while maintaining stable decoding throughput.

Retrieval-augmented prompts are first segmented into fixed-size semantic chunks. To preserve structural integrity, protected regions such as the instruction prefix, system-style control text, and suffix answer anchors are excluded from pruning. For the intermediate evidence region, we implement a first-order gradient analysis to detect a "relevance cliff." Let $s_1 \geq s_2 \geq \cdots \geq s_n$ denote the sorted relevance scores of retrieved chunks. The compressor identifies the optimal cutoff $k = \min\{i \mid |s_i - s_{i+1}| \geq \tau\}$, where τ is an empirical threshold set to 0.1 from a validation slice by selecting the smallest gap value that consistently separated stable high-density evidence clusters from the low-value tail. If no sharp cliff is detected, the system defaults to an adaptive retention policy (70% for high-density clusters vs. 30% for sparse tails). This methodology approximates direct KV-cache management by reducing the allocation state before it enters the weights-attention bottleneck, without requiring invasive modifications to the inference engine.

3.5 Telemetry and orchestration

All major execution stages are instrumented through a unified orchestration layer. The system records retrieval time,

scoring time, compression time, TTFT, decode speed, wall-clock total latency, and concurrency-related tail latency. The same telemetry schema is retained across backend comparison, keep-ratio scans, and concurrency tests so that stage-wise measurements remain comparable even when experiment outputs differ. This telemetry design enables fairer comparison across execution paths and supports bottleneck analysis.

The role of the orchestration layer is not only to collect metrics, but also to impose a common comparison protocol across heterogeneous experiments. Backend comparisons, keep-ratio scans, and concurrency tests do not produce the same raw outputs, so a shared telemetry layer is needed to normalize which signals are retained and how they are interpreted. In the current study, the most important normalized outputs are stage-wise latency, aggregate wall time, and the distinction between prefill-dominated and decode-dominated behavior.

This design also supports one of the paper's broader methodological claims: negative results are meaningful only when their failure mode is visible. For example, the `Falcon-7B + MPS` route is not useful merely because it is slow; it is useful because telemetry shows that the slowness occurs before a stable local baseline can even be established. Without stage-wise instrumentation, this result would collapse into a single large latency number and would be much harder to interpret as evidence about backend choice.

4 Experimental Setup

4.1 Experimental objectives

The experiments address three questions: whether backend choice creates order-of-magnitude performance differences on constrained UMA hardware; whether retrieval compression and context compression further improve TTFT and throughput once generation becomes the main bottleneck; and whether these optimizations introduce unacceptable quality degradation.

4.2 Hardware and runtime environment

All core experiments are conducted on a Mac mini M4 with 16GB unified memory. This environment is particularly suitable for studying local deployment bottlenecks because weights, KV cache, and system processes all share the same memory bandwidth pool. The evaluated deployment paths include:

- `Falcon-7B` on `MPS` as a negative-result baseline
- `Falcon-7B-Quant` on GGUF as a quantized path
- `Llama-3-8B-Quant` on GGUF as the preferred path

This hardware choice is not incidental. The Mac mini M4 platform provides a concrete example of the deployment regime addressed by the paper: a consumer device that is powerful enough to run local 7B- to 8B-class models, yet constrained enough that memory layout and prompt budget remain first-order concerns. The experiments therefore do not aim to establish universal ranking across all accelerators. Instead, they characterize how different optimization routes behave when weights, KV cache, retrieval state, and operating-system processes draw from the same unified resource pool.

4.3 Experimental groups

The experiments are organized into four groups. The first group compares backend-model pairs in order to determine whether the system is practically executable at all. The second evaluates TurboQuant as a retrieval-side compression method and measures the trade-off between retrieval efficiency and ranking fidelity. The third scans keep-ratio-based context compression to determine whether prompt-side reduction improves responsiveness primarily through TTFT rather than through decode throughput. The fourth subjects the optimized path to concurrent load in order to assess whether collaborative optimization improves only single-query latency or also stabilizes tail behavior.

These groups are intentionally sequential. The backend comparison establishes a feasible execution substrate. Retrieval compression and context pruning are then evaluated as optimizations within that feasible substrate rather than as independent techniques in search of a benchmark. Finally, concurrency and quality-preservation checks are used as deployment-oriented validation layers. This ordering aligns the experimental design with the actual engineering process of building a usable local RAG pipeline.

4.4 Metrics

The main metrics used in this study are TTFT, TPS, wall time, P95 TTFT where explicitly stated, Recall@k, MAE, Max Error, and merged dual-judge Faithfulness/Relevance. This metric design follows a practical evaluation order: first determine whether the system is executable, then identify where the bottleneck lies, and finally examine whether the resulting quality loss is acceptable.

Retrieval-side metrics measure the price of compression in index fidelity. For end-to-end output preservation, an automated dual-judge system provides an acceptability check on the final answer behavior. The dual-judge pipeline uses two strong teacher models (`Qwen3-8B` and `DeepSeek-R1-8B`) as independent evaluators to score Faithfulness and Relevance, and reports merged scores on a 0–10 presentation scale after averaging the two judge outputs. By separating these signals, the evaluation avoids treating all improvements as interchangeable. Unless otherwise stated, repeated backend and concurrency conditions use `n = 5` runs, expanded long-context checks use `n = 3` rounds per depth, and answer-level dual-judge comparisons use the `test40` slice (`n = 40`).

This isolation enables us to profile application-level proxy signals for hardware pressure, which are further cross-validated against OS-level virtual memory paging monitors. These measurements provide operational evidence of hardware stress, but they should not be interpreted as direct PMU-level proof of memory-bus saturation.

4.5 Data recording and interpretation policy

The benchmark assets used in this paper are heterogeneous in nature, so a layered interpretation policy is adopted. Multi-run scans and stress tests are reported using mean values, proportional change, and trend interpretation. Single-run but high-value system observations, such as the extreme slowness of `Falcon + transformers + MPS`, are reported as negative-result evidence. Quality interpretations prioritize the merged dual-judge view to reduce reliance on a single judge model.

This policy is necessary because the paper combines measurements of different granularity. A keep-ratio sweep naturally yields a small response curve; a concurrency run produces aggregate stress statistics; and a failed deployment path may offer only one meaningful observation before becoming impractical. The manuscript therefore distinguishes between trend evidence, comparative evidence, and cautionary evidence. This distinction helps prevent overclaiming while still allowing the paper to retain practically important system observations that would otherwise be discarded.

5 Results

5.1 Backend and quantized deployment comparison

RQ1 asks whether backend selection alone can recover practical deployability on constrained UMA hardware. This experiment therefore compares TTFT and decode throughput across different backend-model combinations on the tested device.

Table 1: Backend and model comparison on a Mac mini M4 with 16GB unified memory. The `MPS` row is retained as a cautionary baseline, whereas repeated statistics (`n = 5`) are reported in the main text for the two GGUF routes only.

Configuration	Backend / Model	TTFT (ms)	TPS	Total Latency (s)	Verdict
Negative	Falcon-7B on MPS	77573.00	0.01	264.25	Impractical
Quantized	Falcon-7B-Q + GGUF	123.71	18.09	3.83	Usable
Preferred	Llama-3-8B-Q + GGUF	51.25	19.27	6.69	Preferred

The most important observation is that backend migration within the same model family recovers deployability on the tested device. The first two rows of Table 1 therefore serve as a near backend-migration comparison inside the `Falcon-7B` family: moving from the `MPS` executor to a `GGUF`-based backend reduces TTFT from `77.6 s` to `123.7 ms` and restores usable `18.09 TPS` decode throughput without changing the underlying model family. The third row, `Llama-3-8B-Quant + GGUF`, is included as a practical reference configuration that shows what a more deployable quantized route looks like under the same device constraints, reaching `51.2 ms` TTFT.

Two additional interpretations follow from this result. First, the most dramatic gain appears before any cross-family model comparison is needed: the backend route alone can determine whether a local path is deployable at all. Second, the negative baseline provides a clear boundary for what counts as deployable on the tested device. Once TTFT reaches minute-like scales and decode throughput collapses toward zero, the route is no longer meaningful as an interactive local RAG backend. This boundary motivates the rest of the paper's optimization sequence.

To move this conclusion beyond a single observation, a follow-up repeated benchmark (`n = 5`) was run on the two GGUF paths. Under that repeated setting, `Falcon-7B-Q + GGUF` reaches `158.486 ± 52.038 ms` mean TTFT and `17.218 ± 0.859 TPS`, whereas `Llama-3-8B-Q + GGUF` reaches `122.888 ± 40.561 ms` mean TTFT and `16.960 ± 0.643 TPS`. Table 1 should therefore be read as reporting representative deployment-level reference runs, whereas the repeated benchmark reports mean performance under rerun conditions that include cold-start and run-to-run variance. These repeated measurements should not be read as converting the `MPS` row into an equivalent statistical comparison condition; the `Falcon-7B on MPS` row remains imported cautionary evidence because that route is operationally impractical in the current paper track. What the repeated check does show is that the deployability of the GGUF routes is stable under reruns rather than a one-off artifact of a single benchmark pass.

5.2 Unified Memory Bandwidth Profiling

To provide direct physical evidence of the memory bottleneck, real-time virtual memory pressure and SSD swap paging frequencies are captured continuously via the native macOS `vm_stat` kernel probe.

As shown in Figure 3, the performance collapse of the unoptimized `MPS` execution route is accompanied by a paging crisis at the operating-system level. In a 16GB UMA system, the combination of full-precision model tensors and unmanaged KV-cache growth can overflow the available DRAM, forcing the macOS kernel to initiate aggressive SSD-based swap paging. Our telemetry, captured via real-time `vm_stat` probes, records sustained swap peaks exceeding `67,000 ops/sec` during the `MPS` prefill phase. By contrast, the optimized pipeline—using quantized GGUF weights and compressed retrieval—maintains a substantially lower paging profile. This should be interpreted as hardware-facing operational evidence consistent with the latency results, rather than as a complete hardware-counter proof of bus saturation.

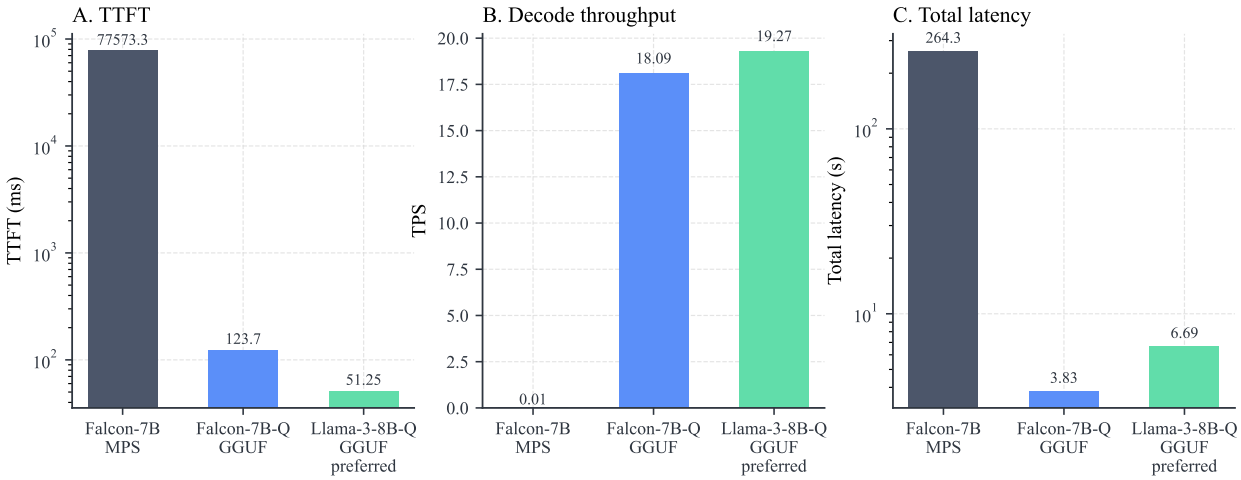

Figure 2: Backend and deployment comparison on the tested 16GB Mac mini M4. Panel A reports TTFT, Panel B reports decode throughput, and Panel C reports total latency. `Falcon-7B` on MPS is included as a negative-result baseline, `Falcon-7B-Q` on GGUF shows the effect of moving to a quantized deployment path, and `Llama-3-8B-Q` on GGUF is shown as the preferred practical configuration.

5.3 Retrieval compression trade-off

RQ2 asks how much retrieval-side footprint can be reduced before ranking fidelity becomes materially degraded. TurboQuant is therefore evaluated under a high-entropy synthetic query setting.

Table 2: TurboQuant retrieval compression trade-off under the current benchmark setting.

Method	Retrieval Latency (ms)	Index Size (MB)	Recall@10	MAE	Max Error
Standard FP32 Distance	128.4	420.0	1.000	–	–
TurboQuant ADC (8-bit)	25.4	110.0	0.631	0.0144	0.0172

TurboQuant reduces retrieval latency by `80.2%` and storage by `73.8%`, but lowers `Recall@10` to `0.631`. This result should not be interpreted as lossless retrieval compression. Rather, it demonstrates an engineering trade-off: retrieval-side memory pressure can be substantially reduced at the cost of partial recall degradation.

For the current paper, the key implication is not that TurboQuant should always replace an uncompressed index. Rather, it offers a controllable way to study how much retrieval-side footprint can be reduced before answer quality becomes meaningfully unstable. In other words, TurboQuant is valuable here as a systems lever: it reduces the cost of carrying retrieval state into the broader execution path, thereby clarifying when retrieval compression helps the full pipeline and when it simply moves error upstream. The method is therefore interpreted here as a retrieval-footprint control mechanism, not as a universally preferable retrieval upgrade.

Table 3 extends this retrieval-centric view to the answer level by comparing an `FP32` retrieval reference against the `TurboQuant` path under a fixed downstream configuration (`top_k = 10`, `keep_ratio = 0.6`). In this end-to-end setting, enabling TurboQuant reduces mean retrieval latency from `111.480 ms` to `69.894 ms` while shrinking the measured retrieval index footprint from `958.5 KB` to `406.4 KB`. At the retrieval level, the corresponding ranking penalty is small: `Recall@10` decreases only from `0.800` to `0.787`. This result is therefore consistent with the systems interpretation that retrieval compression can release meaningful execution budget without inducing a catastrophic ranking collapse.

Table 3: End-to-end TurboQuant ablation on `test40` (`n = 40`, `top_k = 10`, `keep_ratio = 0.6`). Scores use the merged dual-judge 0–10 presentation scale; `Footprint`, `Retr.`, `Faith.`, `Relev.`, and `Err.` denote footprint, retrieval latency, merged faithfulness, merged relevance, and answer error rate, respectively.

Path	Footprint	Retr.	R@10	Faith.	Relev.	Err.
FP32 ref.	958	111.5	0.800	4.81	5.33	0.50
TurboQuant	406	69.9	0.787	4.61	5.14	0.55

However, the answer-level trade-off is not zero. Under the same test slice, merged faithfulness decreases from `4.812` to `4.612` and merged relevance decreases from `5.325` to `5.138`, while answer error rate rises slightly from `0.50` to `0.55`. The manual audit is especially informative here: it does not show an obvious increase in `retrieval_miss` cases (`6` vs. `5`), but it does show more `factual_drift` cases under TurboQuant (`14` vs. `17`). The most defensible interpretation is therefore that TurboQuant acts as a bounded-cost retrieval-footprint reduction lever whose downstream penalty is expressed more through answer-support instability than through simple top-`k` retrieval failure.

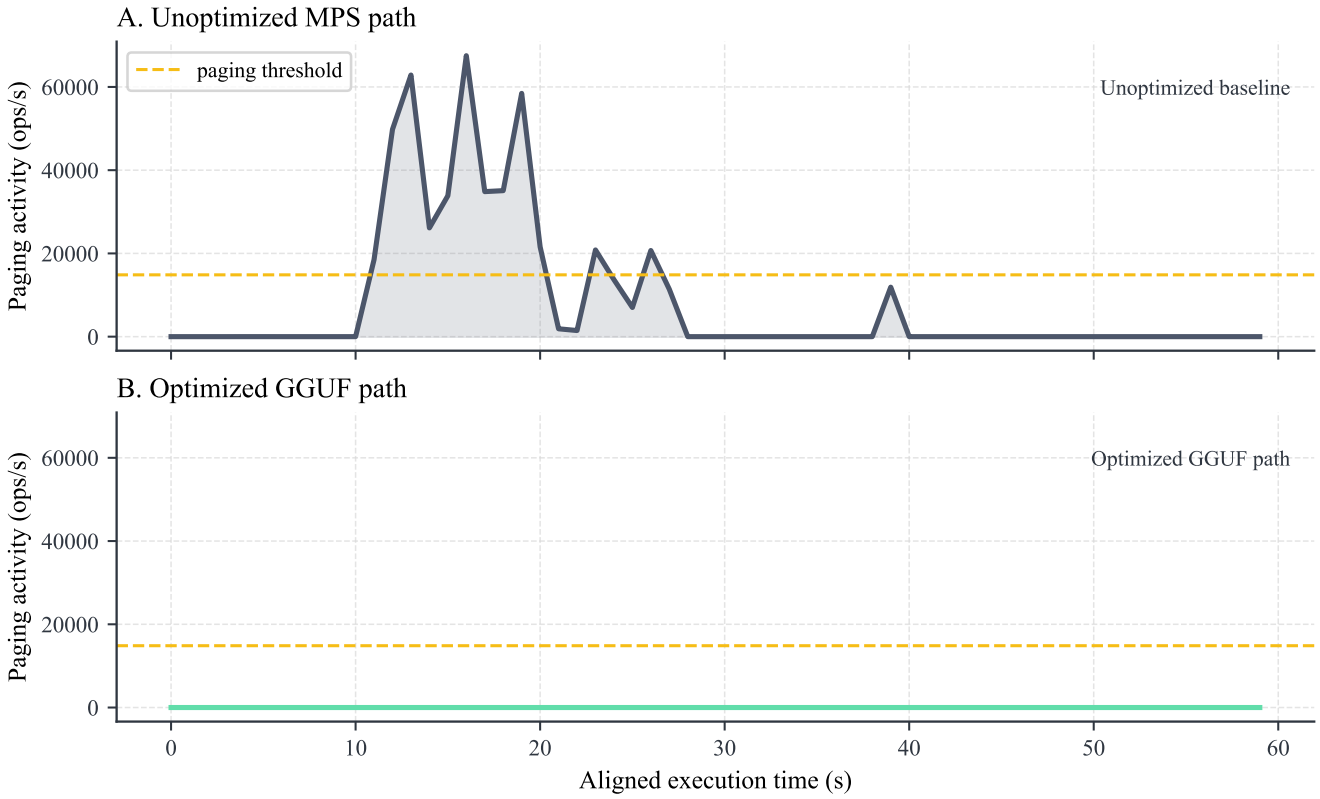

Figure 3: OS-level paging activity recorded during representative benchmark runs on the tested 16GB unified-memory device. The MPS baseline exhibits sustained high paging intensity during the critical execution window, whereas the optimized GGUF-based pipeline reduces paging substantially. This comparison links latency behavior to physical memory-pressure events rather than to wall-time measurements alone.

5.4 Context pruning and TTFT

To examine whether prompt-side reduction improves generation responsiveness, keep ratios from `1.0` to `0.2` are scanned.

In the coarse scan shown in Figure 4, the most favorable prompt-side latency appears near `keep_ratio = 0.6`. A denser follow-up scan over `0.1` increments returns the selected candidate to the same keep ratio under the current setting, which strengthens the interpretation of `0.6` as a local operating point rather than overturning it. At the same time, that denser sweep exhibits large TTFT variance, consistent with cold-start sensitivity and first-run noise, so the selected point should still be read as trend-supported operating-point evidence rather than as a universally optimal pruning ratio.

More broadly, the results show that context pruning primarily improves prefill rather than decode throughput. Decode TPS remains materially more stable than TTFT across the scan, while prompt-side latency changes substantially. This pattern is important because it validates the intended role of context pruning. If both TTFT and TPS had shifted sharply, the result would be harder to attribute cleanly. Instead, the relative stability of decode throughput alongside the movement in TTFT supports the interpretation that keep-ratio pruning mainly targets prompt-side cost. The existence of a non-monotonic optimum around `0.6` also suggests that overly aggressive compression can give back some of the latency gains by distorting prompt structure or reducing useful contextual support.

Table 4: Component ablation summary. Removing context pruning mainly hurts prompt-side latency, whereas removing TurboQuant mainly hurts retrieval cost and index footprint.

Drop	Metric	Full	W/o	Effect
CP	TTFT (ms)	874.3	5828.5	+566.7%
CP	Total E2E (ms)	14612.6	20270.8	+38.7%
TQ	Retrieval (ms)	25.4	128.4	+405.5%
TQ	Index size (MB)	110	420	+281.8%
TQ	Recall@10	0.631	1.000	+58.5%

Table 4 provides a compact ablation view of the two core execution optimizations. Removing context pruning primarily hurts prompt-side latency: TTFT increases from `874.3 ms` to `5828.5 ms`, and total end-to-end latency rises by `38.7%`. Removing TurboQuant primarily hurts retrieval efficiency and footprint: retrieval latency increases from `25.4 ms` to `128.4 ms`, while index size grows from `110 MB` to `420 MB`. The same ablation also restores `Recall@10` from `0.631` to `1.000`, which makes the trade-off explicit

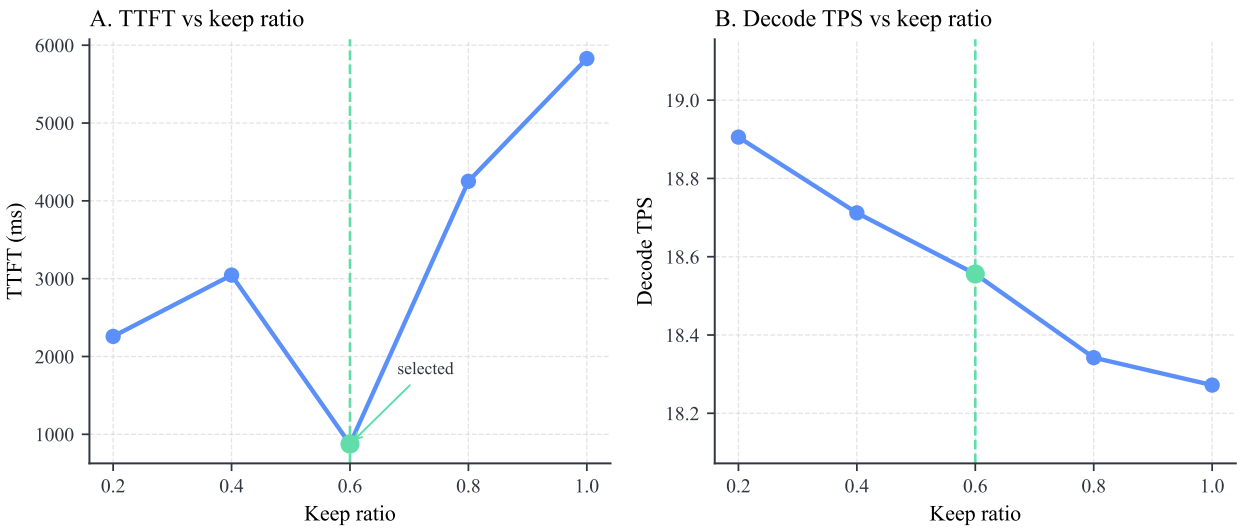

Figure 4: Effect of keep-ratio pruning on prompt-side latency and decode throughput. Panel A shows the change in TTFT across pruning levels, and Panel B shows the corresponding decode TPS. The lowest TTFT is observed near `keep_ratio = 0.6`, while throughput remains comparatively stable, supporting the interpretation that pruning mainly reduces prefill cost on the tested device.

rather than hidden.

5.5 System stability under concurrency

The concurrency experiments compare the full-context path with the collaboratively optimized path under multi-request load.

Table 5: Concurrency behavior under full-context and optimized execution paths at two contention levels, averaged over `n = 5` repeated rounds at each concurrency level. Success rate is computed as completed requests divided by submitted requests across repeated rounds; mean errors/run reports the average number of failed requests per round.

Mode / N	Success rate	P95 TTFT (ms)	Aggregate TPS	Wall time (s)	Mean errors/run
Full context / 2	1.00	7725.955	6.352	40.670	0.0
Optimized path / 2	1.00	6921.581	7.025	36.818	0.0
Full context / 4	0.95	13200.869	4.613	106.391	0.2
Optimized path / 4	0.75	10369.998	5.408	76.382	1.0

Table 5 reports success-aware concurrency results at `N=2` and `N=4`. Here, success rate is computed as completed requests divided by submitted requests across repeated rounds, and mean errors/run reports the average number of failed requests per round. At `N=2`, the optimized path improves tail latency without introducing additional failures: P95 TTFT falls from `7725.955 ms` to `6921.581 ms` (`-10.4%`), aggregate throughput increases from `6.352` to `7.025` tokens per second (`+10.6%`), and wall time drops from `40.670 s` to `36.818 s` (`-9.5%`), while both paths retain a `1.00` success rate.

At `N=4`, the latency and throughput gains become larger, but the stability cost also becomes explicit. The optimized path reduces P95 TTFT from `13200.869 ms` to `10369.998 ms` (`-21.4%`), raises aggregate throughput from `4.613` to `5.408` tokens per second (`+17.2%`), and lowers wall time from `106.391 s` to `76.382 s` (`-28.2%`). At the same time, its success rate decreases from `0.95` to `0.75`, and the observed error count rises from `0.2` to `1.0`. The appropriate interpretation is therefore not that the optimized path is uniformly more stable, but that it improves throughput-oriented efficiency and tail latency under contention while still exposing backend fragility at higher concurrency.

This result strengthens the paper's systems argument. A local path that appears responsive in single-query testing may still degrade badly when multiple requests compete for the same unified-memory bandwidth. The concurrency result therefore serves as a deployment-oriented validation step: collaborative optimization alleviates tail latency and completion time under contention, but it does not remove the backend stability boundary once concurrency increases.

5.6 Long-context robustness

To complement the latency-oriented experiments, a lightweight long-context robustness check is retained under a `120k`-character needle-in-a-haystack probe [6]. Rather than increasing only the number of benchmark documents, this test asks whether the optimized retrieval-and-pruning path still recovers a target fact when the context becomes substantially longer and the target moves toward the end of the haystack.

Table 6 expands the depth sweep to `0%`, `25%`, `50%`, `75%`, `90%`, `95%`, and `99%`. Robustness remains high at `0%`, `25%`, `50%`, and `99%`, where the success rate is `1.00`; by contrast, `75%`, `90%`, and `95%` drop to `0.67`, with failures dominated

Table 6: Expanded 120k-character needle-in-a-haystack depth sweep, reported as bounded robustness evidence with n = 3 rounds per depth.

Needle depth (%)	Success rate	Latency (s)	Failure
0	1.00	11.90	–
25	1.00	11.12	–
50	1.00	11.20	–
75	0.67	11.18	missed
90	0.67	9.88	missed
95	0.67	9.48	partial
99	1.00	11.25	–

by missed or partial cases. Mean latency stays in a relatively narrow 9.48–11.90 s band across the sweep. The expanded depth check therefore strengthens the interpretation of this section as a bounded robustness signal rather than as a guarantee of stable long-context scaling across all deeper needle positions.

5.7 Quality preservation under compression

To avoid interpreting the paper as speed-only optimization, quality preservation is examined using dual-judge test40 results. This section differs from Section 5.3 in evaluation scope: Table 3 isolates the effect of retrieval compression under a fixed downstream setting, whereas the current section evaluates the final selected operating point after bounded joint selection over topn × budget under the current paper-track protocol.

Table 7: Quality preservation under the selected operating point. The table compares the baseline path with the final operating point from the bounded topn × budget search region using TTFT, total latency, and merged dual-judge Faithfulness/Relevance scores on the test40 evaluation split (n = 40).

Config.	TTFT (ms)	Total latency (ms)	Merged faithfulness	Δ Faithfulness	Merged relevance	Δ Relevance
Baseline	2026.509	7844.002	4.237	—	4.975	—
Selected	1344.748	7915.441	4.500	+0.263	5.112	+0.137

As shown in Table 7, the selected operating point reduces TTFT from 2026.509 ms to 1344.748 ms, corresponding to a 33.6% improvement in prompt-side responsiveness. By contrast, total latency changes only slightly, increasing from 7844.002 ms to 7915.441 ms (+0.9%). This divergence is consistent with the broader systems interpretation of the paper: once selection is driven primarily by prompt-side responsiveness within a bounded acceptable region, total latency can remain decode-dominated and need not improve in lockstep.

In the same comparison, merged faithfulness increases from 4.237 to 4.500, and merged relevance increases from 4.975 to 5.112. Figure 5 summarizes this prompt-side latency-quality shift from the baseline to the selected operating point. These results indicate that the final selected operating point improves prompt-side responsiveness while remaining inside the bounded acceptable region on the current evaluation slice. The appropriate interpretation is therefore not that the selected operating point is globally optimal across all objectives, but that the current optimization path identifies a practically acceptable operating region in which TTFT can be reduced without obvious answer-quality collapse.

Table 8: Surviving candidates in the bounded topn × budget region (n = 40).

Candidate	TTFT (ms)	Total (ms)	Faith.	Relev.	Sel.
t5_b1500	1344.748	7915.441	4.500	5.112	Yes
t10_b2000	1983.836	8005.640	4.950	4.938	No
t10_b2500	2446.941	8739.499	5.112	5.157	No

The selected operating point was not chosen ad hoc; rather, it was selected from a bounded topn × budget search region evaluated under the same current paper-track generation and dual-judge protocol. Table 8 makes the three surviving candidates explicit. Among them, t5_b1500 was retained as the final operating point because it offered the lowest TTFT while remaining inside the acceptable region, whereas the two topn=10 alternatives represented higher-quality but slower operating points.

Under the current evaluation setting, the selected operating point should be understood as a bounded operating point rather than as a globally optimal one: it improves TTFT and slightly improves merged answer-quality scores on the present slice, but it does not simultaneously minimize total latency or all failure proxies.

5.8 End-to-end latency decomposition

The decomposition results make the bottleneck migration explicit for the baseline keep_ratio = 1.0 path and the optimized keep_ratio = 0.6 path shown in Figure 6. End-to-end latency falls from 20.3 s to 14.6 s, with the largest absolute reduction coming from prefill (5.29 s to 0.87 s). Decode changes much less in absolute terms (14.44 s to 13.69 s) and therefore remains the dominant residual cost in the optimized path.

Table 9: Representative stage-wise latency of the optimized path at keep_ratio = 0.6. Decode remains the dominant residual cost.

Stage	Time (ms)	Share (%)
Retrieval	21.1	0.14
Compression	28.9	0.20
Prefill	874.3	5.98
Decode	13688.3	93.67
Total	14612.6	100.00

Table 9 reports the representative stage-wise latency of the optimized path at keep_ratio = 0.6. Here, "compression" combines scoring and pruning time. The table makes

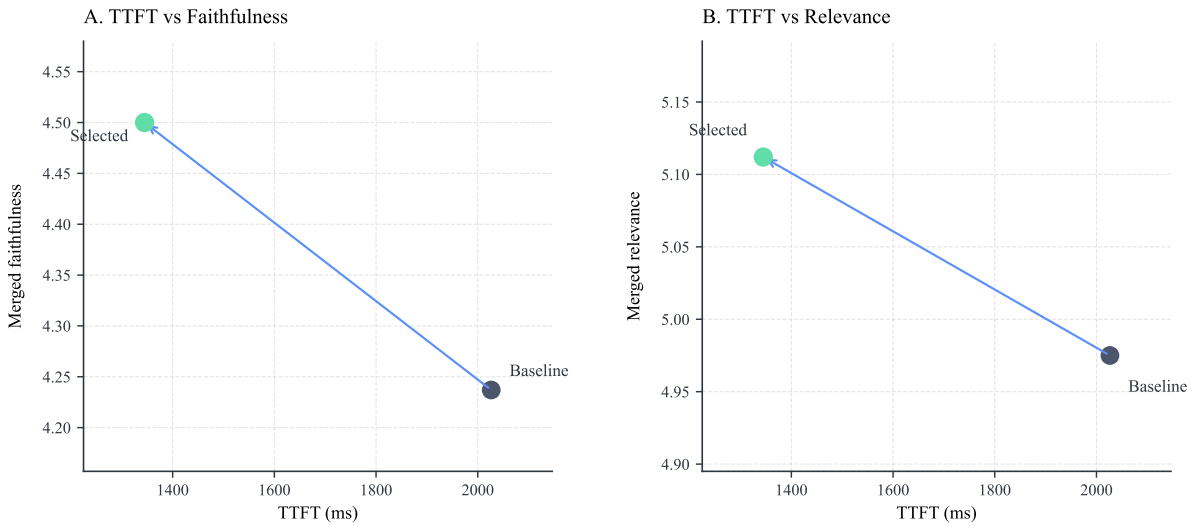

Figure 5: Prompt-side latency-quality shift from the baseline to the selected operating point. Panel A shows that the selected operating point moves to a lower-TTFT region while improving merged faithfulness. Panel B shows that the same shift is accompanied by a small increase in merged relevance. The operating point shown here corresponds to the final operating point selected from a bounded `topn` × `budget` search region under the same current paper-track generation and dual-judge protocol.

the main practical point explicit: retrieval and compression are already in the tens-of-milliseconds range, prefill has been reduced below one second, but decode still consumes `93.7%` of the remaining latency budget. This is the direct reason why backend tuning and prompt-budget control remain more consequential than further isolated retrieval-side refinements.

This decomposition closes the loop on the paper's framing. Earlier sections argue that local RAG on constrained hardware should be treated as a systems co-design problem. Figure 6 shows why: retrieval and context-side interventions meaningfully reduce cost, yet once these gains accumulate, the remaining latency mass is concentrated in generation, especially decode. The optimization target therefore migrates rather than disappears. This is precisely why a paper framed only around retrieval-side improvement would be incomplete for the tested deployment regime.

6 Discussion

The primary contribution of this study is the demonstration that local RAG performance on constrained unified-memory hardware should be interpreted as a shared-budget systems problem. Unlike cloud-based systems where high-throughput accelerators can hide stage-wise memory interactions, UMA devices such as the Apple Silicon M4 enforce a visible resource ceiling. The empirical results indicate that reducing retrieval or prefill overhead can release practical execution budget for the LLM's autoregressive decode phase, but the benefit remains bounded by downstream generation cost. This broader framing is consistent with KV-state management work such as H_2O, which likewise treats inference-time state retention as a first-order systems bottleneck rather than a secondary implementation detail [11].

6.1 Memory saturation and paging-dominated execution failure

The performance collapse observed in the unmanaged `Falcon-7B + MPS` execution route (Section 5.1) highlights a fundamental systems failure mode on constrained hardware. In the tested setting, this failure mode appears as a paging-dominated execution regime in which model execution triggers sustained SSD-backed paging and rapidly loses interactive utility. As weights and KV-cache activations exceed the 16GB DRAM pool, the system can be pushed into that regime. Our recorded peak swap frequency of `67,000 ops/sec` (Figure 3) provides an operational signature consistent with this interpretation. In this state, even a theoretically stronger model becomes practically unusable because paging overhead dominates the compute budget. The optimized framework reduces the severity of this failure mode by constraining the operational footprint through GGUF quantization, retrieval compression, and prompt-budget control rather than eliminating it entirely.

6.2 Bottleneck Migration and Staged Optimization

A critical finding of this study is the phenomenon of *bottleneck migration*. Our stage-wise decomposition (Section 5.8) shows that as retrieval and prefill costs are reduced through compression and pruning, the system's dominant constraint migrates from bandwidth-sensitive setup cost toward gener-

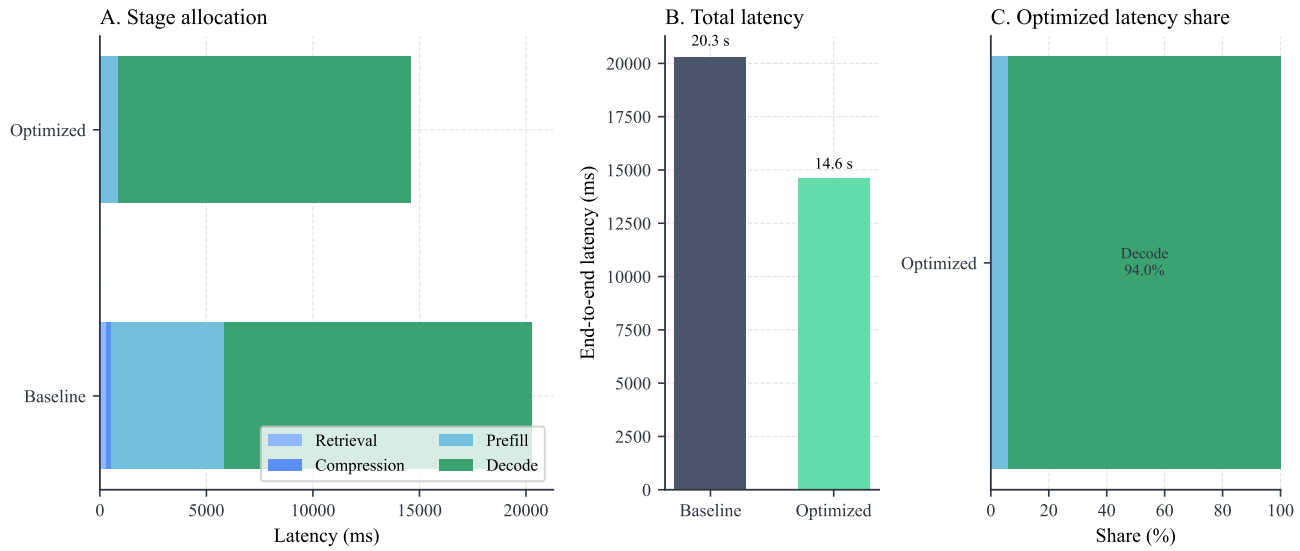

Figure 6: Stage-wise latency decomposition before and after applying the optimized execution path on the tested 16GB unified-memory device. Panel A compares latency allocation across retrieval, scoring/compression, prefill, and decode stages; Panel B summarizes the end-to-end reduction; and Panel C shows the residual latency share after optimization. The figure highlights that, once retrieval and prefill costs are reduced, decode remains the dominant bottleneck.

ation. Specifically, once prefill latency is reduced from seconds to sub-second levels, the remaining `93.7%` of end-to-end latency is concentrated in the decode loop.

This leads us to propose a *Staged Optimization Hierarchy* for practitioners deploying local RAG:

1. **Substrate Recovery**: Select an inference backend (e.g., GGUF/llama.cpp) specifically designed for hardware-aware memory mapping and quantization to restore a functional baseline.
2. **Prefill Budgeting**: Apply context-aware pruning to reduce input sequence length. This targets the $\mathcal{O}(L^2)$ attention complexity of prefill, which is the primary source of initial interaction lag.
3. **Retrieval Footprint Reduction**: Deploy vector-side compression (e.g., TurboQuant) to shrink the retrieval state, ensuring that index-loading does not trigger early memory pressure.

6.3 Collaborative Optimization vs. Modular Isolation

Mainstream RAG research frequently optimizes retrieval (e.g., better rerankers) or serving (e.g., PagedAttention) in isolation. However, our findings suggest that on constrained hardware, modular isolation is a suboptimal design pattern. Because all stages share the same physical bandwidth pipe, an improvement in retrieval that increases the retrieved context length may inadvertently degrade generation latency by ballooning the KV-cache footprint. Our collaborative framework treats the entire execution path as a single pipeline, where modular decisions are made based on their collective impact on the hardware's shared resource limits. On constrained UMA hardware, the relevant design objective is therefore not the strongest isolated module, but the best end-to-end resource allocation across retrieval, prefill, and decode.

The updated concurrency evidence sharpens this point. Under load, the optimized path continues to reduce tail latency, wall time, and memory-pressure severity, yet it also shows a lower success rate at `N=4`. Collaborative optimization therefore improves efficiency under contention, but high-concurrency execution remains bounded by backend robustness and unified-memory pressure.

The quality-preservation results further refine this interpretation. In the selected operating point, TTFT decreases substantially while total latency remains roughly flat and slightly higher, which is consistent with a decode-dominated residual cost structure. At the same time, both merged faithfulness and merged relevance increase on the present evaluation slice. This pattern suggests that the current optimization path operates within a bounded acceptable region rather than providing cost-free acceleration across all quality dimensions. Accordingly, the selected operating point should be understood as the most balanced survivor under the current protocol, not as a claim that one configuration simultaneously optimizes every latency and quality objective.

The later bounded `topn` × `budget` search further supports this interpretation. The final selected operating point is not an arbitrary choice, but the most balanced surviving candidate within a small search region evaluated under the same paper-track protocol. Nearby alternatives such as `t10_b2000` and `t10_b2500` improve merged answer quality only by paying materially higher prompt-side la-

tency, which makes the quality-latency trade-off explicit rather than implicit.

A later pruning-policy ablation supports the same operating-point interpretation from the prompt-budget side. More aggressive or simpler pruning policies can reduce TTFT further, but they do so at a clearer answer-level quality cost, leaving the static `keep_ratio = 0.6` policy as the more balanced operating point under the current protocol.

The complementary FP32-reference versus TurboQuant-enabled ablation supports the same bounded-cost interpretation from the retrieval side. Under a fixed generation setting, TurboQuant substantially reduces retrieval latency and index footprint while only slightly reducing `Recall@10`; yet answer-level merged faithfulness and relevance also decline modestly, and the audit records more `factual_drift` cases rather than a clear rise in `retrieval_miss`. This indicates that retrieval compression should not be presented as a lossless upgrade. Its practical value lies in releasing shared execution budget at a measurable but limited answer-level cost.

The results also imply a practical deployment order. If a local path is not yet interactively usable, backend migration is the first priority because it determines whether a stable baseline exists at all. Once that baseline is restored, prompt pruning is the highest-leverage intervention for reducing TTFT. Retrieval compression becomes most useful after those steps, when the remaining question is how much retrieval-side footprint can be traded away without unacceptable recall or answer-quality loss.

7 Threats to Validity

This study has several limitations that bound the generalizability of our findings. First, the conclusions are primarily derived from a single hardware class (Mac mini M4, 16GB UMA). While the unified memory principles discussed are broadly applicable to the Apple Silicon ecosystem, results may differ on discrete-GPU architectures (e.g., NVIDIA RTX) where dedicated VRAM bandwidth frequently exceeds unified DRAM speeds. Second, our experiments focus on 7B- to 8B-class models; larger models (e.g., 70B) may exhibit different bottleneck patterns where weight-transfer latency significantly outweighs KV-cache growth even under collaborative optimization.

There is also a scope limitation in our quality assessment. While our dual-judge framework provides a reliable signal for detecting faithfulness collapse, it remains a proxy for human expert evaluation and large-scale task-specific benchmarks. Similarly, our concurrency experiments reflect system stability under synthetic load; real-world multi-user performance may be affected by unpredictable application-level cold-start overhead or OS-level thermal throttling not captured in our controlled lab setting.

Moreover, the quality-preservation conclusion should be interpreted as bounded evidence for the current evaluation slice and scoring protocol rather than as a task-general guarantee that latency-oriented compression leaves all answer-quality dimensions unchanged.

Finally, we acknowledge that the layered nature of our evidence carries varying degrees of strength. The trend-based findings for keep-ratio pruning and TurboQuant trade-offs are supported by repeated, structured scans. Conversely, the negative-result `MPS` baseline is retained as a case-study observation of hardware saturation rather than a statistical trend. We have intentionally distinguished between these evidence classes throughout the manuscript to provide a transparent account of the system's operational boundaries.

8 Reproducibility Statement

All major conclusions in this paper are bound to existing project assets, including benchmark JSON files, research logs, and execution scripts. Core evidence comes from backend-comparison logs, performance scans, keep-ratio trade-off curves, concurrency stress reports, quantization-quality evaluations, and the project research log. The current review artifact bundle provides model identifiers, quantization formats, runtime flags, script entry points, and a compact result manifest for the reported claims.

At a minimum, the artifact package should expose three layers of material. The first layer is the benchmark evidence itself, including the backend comparison JSON, the keep-ratio scan outputs, the concurrency stress reports, and the quality-preservation records used in the tables and figures. The second layer is the generation pipeline for derived artifacts, including the plotting script, table sources, and manuscript source used to rebuild the reported figures and tables. The third layer is execution metadata, such as hardware description, backend flags, quantization format, and the command entry points needed to rerun each benchmark family.

This structure is important because the paper's argument depends on stage-wise interpretation rather than only final metrics. Reproducibility therefore requires not only the end tables, but also the intermediate stage records that justify claims about prefill, decode, retrieval compression, and concurrency behavior. The accompanying artifact bundle is designed with this requirement in mind and organizes the manuscript, figure sources, table sources, and evidence files so that each major claim can be traced back to a concrete project asset. The current submission package additionally includes `artifact_manifest.md`, which records model identifiers, quantization paths, runtime configuration fields, and the raw result file paths used by the manuscript.

9 Conclusion

This artifact-backed empirical study examines collaborative optimization for local RAG on constrained unified-memory hardware. On the tested device, once retrieval-side efficiencies are realized, the dominant bottleneck shifts toward gen-

eration, especially decode. The artifact-backed evidence indicates that interactive behavior can be recovered through an integrated approach spanning quantized deployment, retrieval compression, and context pruning, while unmanaged routes remain operationally fragile because of paging pressure. More broadly, the study suggests that local LLM performance on consumer hardware is better evaluated as a coordinated systems problem than as an isolated retrieval task.

Data and Code Availability

The experimental conclusions in this manuscript are derived from local benchmark artifacts, figure-generation scripts, table sources, and research logs maintained in the project workspace. A review artifact bundle is included with the submission package and contains the benchmark JSON files, plotting scripts, table sources, manuscript source, artifact manifest, and a minimal README describing the execution environment and expected outputs. The current public repository for the paper-track materials is available at `https://github.com/deletezero08/local-rag-uma-artifact-study`. A persistent archival identifier will be added in the final archival release.

Acknowledgements

This manuscript builds on a broader engineering effort on local RAG systems under constrained hardware, including prior work on routing, retrieval, and memory management in a related undergraduate thesis project. The current paper focuses on the generation-side optimization path that emerged from those earlier system observations.

Competing Interests

The author declares no competing interests.

Ethics Statement

This study does not involve human subjects, protected personal data, or clinical decision-making. The reported experiments focus on system performance, deployment trade-offs, and quality-preservation checks for local language model execution under constrained hardware conditions.

References

[1] Gordon V Cormack, Charles L A Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.

[2] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Qianqian Guo, Meng Wang, et al. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.

[3] ggml-org. Gguf format tools and documentation. GitHub repository documentation, 2026. Accessed 2026-03-31.

[4] ggml-org. llama.cpp: LLM inference in C/C++. GitHub repository, 2026. Accessed 2026-03-31.

[5] Awni Hannun, Jain Jain, Vineel Garg, Spyros Gidaris, and Adrien Gaidon. Mlx: Efficient and flexible machine learning on apple silicon. *arXiv preprint arXiv:2312.06545*, 2023.

[6] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. Ruler: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

[7] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Llmlingua: Compressing prompts for accelerated inference of large language models. *arXiv preprint arXiv:2310.05736*, 2023. Conference version at EMNLP 2023.

[8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023.

[9] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[10] Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. Turboquant: Online vector quantization with near-optimal distortion rate. *arXiv preprint arXiv:2504.19874*, 2025. Conference version at ICLR 2026.

[11] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H_2o: Heavy-hitter oracle for efficient generative inference of large language models. *arXiv preprint arXiv:2306.14048*, 2023.